EdgeLUT-Agent: A Lightweight Graph-Aware LUT Agent for On-Device Image Restoration

Anonymous Authors
Anonymous Institution
anonymous@anonymous.edu

Abstract

Recent LLM/agent-based image restoration systems chain multiple pretrained foundation models, exceeding 100M parameters and 500 ms CPU inference time, which makes them infeasible for real-time on-device deployment. We present **EdgeLUT-Agent**, a miniature agentic restoration pipeline that uses three Dynamic 3D LUTs as primitive operators, each modulated by a FiLM-conditioned MLP. A shallow perception network estimates per-degradation intensities, a graph-aware threshold router selects and orders the LUTs, and a closed-loop feedback stage triggers at most one re-routing when post-hoc statistics violate quality thresholds. The entire system contains only **48,478 parameters** ($\sim$194 KB in FP32) and runs at **0.99 ms CPU latency** per 28×28 image, while achieving **26.78 dB PSNR / 0.949 SSIM** on a composite-degradation MNIST benchmark. We further deploy Qwen2.5-0.5B-Instruct as an on-device router: it matches the rule-based router in PSNR (23.64 vs. 23.73 dB) but is **64,000$\times$ slower** (6,412 ms vs. $<$0.1 ms), and exhibits an output-collapse phenomenon in which the small model memorizes a few-shot JSON template rather than reasoning over the numeric observations.

1 Introduction

Image restoration (IR) has recently been reframed as an agentic problem: a large language model (LLM) perceives a degraded image, reasons over a catalog of pretrained restoration experts, and chains them in a learned order [4, 5]. While powerful, this design inherits the failure mode of every model-chaining system on which it is built—parameter count and inference time scale linearly with the number of experts. A typical IR agent today invokes three to five pretrained models of $\geq$30M parameters each, exceeds 100M parameters and 500 ms of CPU latency per image, and therefore cannot run at interactive frame rates on a phone.

Our design philosophy is the opposite of model-chaining: *compress the "intelligence" into a routing decision and offload the actual restoration to extremely lightweight lookup tables.* Concretely, the "restoration" in our system is carried out by three Dynamic 3D LUTs, each a $17^3 \times 3$ grid of learnable entries modulated by a tiny FiLM-conditioned MLP. The "agent" loop is preserved in full: perceive (perception network estimates three degradation intensities), reason (graph-aware threshold router selects and orders operators), act (trilinear-interpolated LUT lookup), and feedback (post-hoc statistics trigger at most one re-route). The result is a closed-loop agentic system whose total budget is $<$ 50K parameters and $<$ 1 ms CPU latency.

Our contributions are:

1. A **Dynamic 3D LUT architecture** in which each LUT is conditioned by a 166-parameter FiLM MLP, giving a per-operator budget of 14,905 parameters and a full-system budget of 48,478.

2. A **graph-aware routing + closed-loop feedback agent loop** that preserves the perceive–reason–act–feedback semantics of LLM agents while keeping per-image CPU latency under 1 ms.
3. A **real deployment study** of Qwen2.5-0.5B-Instruct as an on-device router, revealing an *output collapse* failure mode: the 0.5B model reproduces the few-shot JSON template regardless of the numeric observations, rather than reasoning over them.
4. A **fully open-source implementation** with an HTML on-device demo, weights, and evaluation logs.

2 Related Work

3D LUT-based image enhancement. Learning 3D LUTs as compact enhancement operators was popularized by Zeng et al. [1], who parameterize an image-adaptive color transform as a regular $R \times G \times B$ lookup grid and apply it via trilinear interpolation. Follow-up work extends the idea to spatio-channel-adaptive LUTs and joint demosaicing. Our Dynamic 3D LUT differs in that the grid is not static: a FiLM conditioner predicts per-channel scale and bias that modulate the base grid at inference time.

Lightweight image restoration. Classical feed-forward denoisers such as DnCNN [2] and IRN [3] demonstrated that small convolutional networks can denoise at parameter budgets orders of magnitude below foundation models. Our work is in this spirit but pushes the operator family toward lookup tables, which admit a particularly cheap CPU inference path.

LLM agents for vision. VisAgent-style systems and LLaVA-based restoration pipelines [4, 6] use LLMs to decompose a restoration request into a sequence of pretrained experts. Qwen2.5 [5] provides a family of small instruction-tuned models targeted at edge deployment. Our work is not an alternative to such agents on server-class hardware; rather, it is an empirical study of whether the smallest of these models can serve as a per-image router on a phone, and what breaks when they cannot.

Edge AI deployment. TensorFlow Lite [8], NCNN [9], and ONNX Runtime [10] are the standard runtimes for on-device inference. Our 0.99 ms latency and 194 KB weight size are within the budgets these runtimes target, and we deliberately choose operations (convolution, trilinear interpolation, scalar thresholding) that map cleanly onto them.

3 Method

3.1 Problem Formulation

We model the observed image as a composite degradation of a clean image x:

$$y = D(x;\theta), \qquad D \in \{\text{noise}, \text{blur}, \text{lowlight}\}, \tag{1}$$

where D may be a composition of one or more primitives. The reconstruction objective is

$$\min_{\theta} \mathbb{E}_{(x,y)} \|\hat{x} - x\|_1, \qquad \hat{x} = f_\theta(y). \tag{2}$$

The novelty of f_θ lies not in the loss but in its structure: f_θ is a closed loop of perception, routing, LUT lookup, and feedback.

3.2 Perception Network

The perception network ϕ_{per} estimates a vector of per-degradation intensities $p = [p_n, p_b, p_l] \in [0,1]^3$:

$$\begin{aligned}&\text{Conv}(3\to 8,\, 3\times 3,\, p=1) \to \text{ReLU} \to \text{MaxPool}(2) \to \text{ReLU}\\ &\to \text{Conv}(8\to 16,\, 3\times 3,\, p=1) \to \text{ReLU} \to \text{MaxPool}(2)\\ &\to \text{Conv}(16\to 16,\, 3\times 3,\, p=1) \to \text{ReLU} \to \text{AdaptiveAvgPool}(1) \to \text{FC}(16\to 3) \to \sigma. \end{aligned} \tag{3}$$

The network has **3,763 parameters** and is supervised by an L1-auxiliary loss on the ground-truth degradation intensities.

3.3 Dynamic 3D LUT

Each restoration operator is a Dynamic 3D LUT. The base grid is a learnable tensor of shape $\text{res}^3 \times 3$ with $\text{res} = 17$ (14,739 entries). A FiLM conditioner MLP

$$\text{MLP}(3 \to 16 \to 6) \tag{4}$$

takes p as input and produces six raw parameters. We split them as

$$\text{scale} = \tanh(\text{raw}_{1:3}) \times 0.5 \in [-0.5, 0.5], \quad \text{bias} = \tanh(\text{raw}_{4:6}) \times 0.1 \in [-0.1, 0.1], \tag{5}$$

and the effective LUT applied at inference is

$$\text{LUT}_{\text{eff}} = \text{LUT}_{\text{base}} \odot (1 + \text{scale}) + \text{bias}. \tag{6}$$

At run time, the output pixel is read from LUT_{eff} by trilinear interpolation. Each LUT has **14,905 parameters** (14,739 base + 166 conditioner).

3.4 Graph-Aware Routing

Given p and a threshold $t = 0.3$, the rule router activates every operator whose intensity exceeds t and executes them in decreasing order of intensity. The same decision is exposed to the LLM baseline through a natural-language prompt:

```
You are a router for an image restoration system.
You have three operators: denoise, deblur, lowlight.
Given degradation observations, decide which operators to run.

Observations:
- noise_level: {p_n:.3f}
- blur_level: {p_b:.3f}
- lowlight_level: {p_l:.3f}
- threshold: 0.3

Rules: Only include operators whose level exceeds threshold.
Order by level descending.
Respond ONLY with JSON: {"actions": [...], "order": "...", "skip": [...]}
```

The rule router parses the threshold directly; the LLM router is expected to parse the same three numbers and emit a JSON action list.

3.5 Closed-Loop Feedback

After executing the routed operators, we compute three cheap image statistics on the output $\hat{x}$: mean luminance, Laplacian local standard deviation (sharpness), and high-frequency energy (noise). Re-routing is triggered when any of the following holds:

- estimated noise $> 0.002 \Rightarrow$ re-run denoise;
- mean luminance $< 0.3 \Rightarrow$ re-run lowlight;
- sharpness $< 0.05 \Rightarrow$ re-run deblur.

At most one re-routing is allowed to keep the loop bounded.

Inference procedure. Given a degraded image y and the three LUTs L_n, L_b, L_l:

S1: Compute $p \leftarrow \phi_{\text{per}}(y)$.
S2: Build $\mathscr{A} \leftarrow \{i : p_i > t\}$ and sort it by p_i descending.
S3: Initialize $\hat{x} \leftarrow y$; for each $i \in \mathscr{A}$, set $\hat{x} \leftarrow L_i(\hat{x}; p)$.
S4: Compute $(s_n, s_l, s_s) \leftarrow \text{stats}(\hat{x})$ (noise, luminance, sharpness).
S5: If $s_n > 0.002$ or $s_l < 0.3$ or $s_s < 0.05$, build a feedback action set $\mathscr{A}'$ and re-run the corresponding LUTs once.
S6: Return $\hat{x}$.

4 Experiments

4.1 Datasets and Protocol

We train on a synthetic composite-degradation variant of MNIST: additive Gaussian noise $\sigma \in [5, 15]/255$, Gaussian blur $k \in \{3, 5, 7\}$, $\sigma_b \in [0.5, 2.0]$, and lowlight gamma $\gamma \in [0.3, 0.7]$. Each training sample uses a random subset of at least one degradation. Train/Val/Test split is 5,000 / 1,000 / 2,000. We also evaluate out-of-domain on the Set12 grayscale benchmark [2] using a sliding window with patch size 28 and stride 14.

4.2 Implementation Details

Optimizer: Adam, learning rate 1×10^{-3}, batch size 64, 40 epochs. Loss:

$$\mathscr{L} = \|\hat{x} - x\|_1 + 0.5\,\|\hat{x}_{\text{all-LUT}} - x\|_1 + 0.1\,\text{MSE}(p, p_{\text{gt}}). \tag{7}$$

All timing is measured on a 4-core CPU with 7.9 GB RAM. The Qwen2.5-0.5B-Instruct baseline runs in FP32 on the same CPU and occupies $\sim$2.5 GB RAM.

4.3 Main Results on MNIST

Table 1 reports PSNR/SSIM on the 2,000-image MNIST test set. The full EdgeLUT-Agent reaches 26.78 dB / 0.949, within 0.12 dB of the always-on serial baseline, at a fraction of the compute.

Table 1: Main results on the MNIST composite-degradation test set ($n = 2{,}000$).

Method	PSNR (dB)	SSIM	Params
Degraded input (baseline)	19.03	0.597	0
Single LUT – denoise	21.29	0.720	18,668
Single LUT – deblur	20.77	0.750	18,668
Single LUT – lowlight	21.52	0.794	18,668
All-3 serial (fixed order)	26.66	0.950	48,478
Static LUT + routing (no FiLM)	21.27	0.766	48,312
Ours (rule router + feedback)	**26.78**	**0.949**	**48,478**

4.4 Ablation Study

Table 2 ablates each component. The FiLM conditioner is the single largest contributor ($+5.51$ dB over static LUT), and closed-loop feedback contributes another $+2.34$ dB; routing itself buys only 0.12 dB over always-on serial, which is the expected behavior on a benchmark where most samples activate multiple operators.

Table 2: Ablation on MNIST test set. ΔPSNR is relative to the full model.

Variant	PSNR (dB)	SSIM	ΔPSNR
Full model	26.78	0.949	—
w/o feedback re-route	24.44	0.909	-2.34
w/o FiLM (static LUT)	21.27	0.766	-5.51
w/o routing (all serial)	26.66	0.950	-0.12

4.5 Qwen2.5-0.5B Router Comparison

Table 3 replaces the rule router with Qwen2.5-0.5B-Instruct running on CPU. The small LLM matches the rule router in average PSNR (23.64 vs. 23.73 dB) but at 6,412 ms of latency, roughly 64,000$\times$ slower than the < 0.1 ms rule router.

Table 3: Rule router vs. Qwen2.5-0.5B-Instruct as the on-device router ($n = 100$).

Router	PSNR (dB)	SSIM	Latency (ms)
Rule-based (threshold $t = 0.3$)	23.73	0.904	< 0.1
Qwen2.5-0.5B-Instruct (FP32, CPU)	23.64	0.898	6,412
Δ (Qwen $-$ rule)	-0.09	-0.006	64,000$\times$ slower

Output collapse. Across 100 test samples, Qwen2.5-0.5B produced nearly identical routing decisions regardless of the numeric observations. Table 4 illustrates the failure on two samples. On Sample 00, the true intensities are $p = [0.748, 0.025, 0.014]$: the correct routing is *denoise only*, which the rule router selects. Qwen, however, emits the exact few-shot example actions `[denoise, lowlight]`, incorrectly activating lowlight despite $p_l = 0.014 \ll 0.3$. On Sample 03, $p = [0.607, 0.800, 0.230]$: the correct routing is *deblur then denoise*. Qwen again emits the same `[denoise, lowlight]` template, missing the dominant deblur operator and injecting an unnecessary lowlight pass, which drops PSNR from 16.14 to 13.94 dB.

Table 4: Two representative router samples revealing output collapse. Qwen reproduces the few-shot JSON template `[denoise, lowlight]` in both cases instead of reasoning over p.

	Sample 00	Sample 03
Predicted p_n, p_b, p_l	0.748, 0.025, 0.014	0.607, 0.800, 0.230
Ground-truth p_n, p_b, p_l	0.877, 0.000, 0.000	0.544, 0.834, 0.475
Rule actions	[denoise]	[deblur, denoise]
Qwen actions	[denoise, lowlight]	[denoise, lowlight]
Rule PSNR / SSIM	29.09 / 0.904	16.14 / 0.672
Qwen PSNR / SSIM	32.56 / 0.970	13.94 / 0.577
Qwen latency (ms)	6,844	13,942

Real image evaluation (Set12). Table 5 reports the Set12 sliding-window results. The MNIST-trained model does not generalize to 256×256 natural photos: the perception network flags all three degradations as high on almost every patch, forcing all three LUTs to run in series and pulling the image statistics toward MNIST. This is a clear domain-gap failure rather than a routing failure, and we report it openly.

Table 5: Set12 average results (12 grayscale images, sliding window 28/14).

Method	PSNR (dB)	SSIM
Degraded input	20.97	0.742
Single LUT – denoise (avg)	$\sim$18.5	$\sim$0.65
Ours (routed, sliding window)	9.80	0.231

4.6 Complexity Analysis

Table 6 summarizes the budget of EdgeLUT-Agent and compares it against typical IR agents built by chaining pretrained experts.

Table 6: Complexity comparison.

Metric	EdgeLUT-Agent	Typical IR Agent
Total parameters	48,478	$>$100M
Weight size (FP32)	$\sim$194 KB	$>$400 MB
FLOPs per image	$\sim$1.09M	$>$10G
CPU latency (measured)	0.99 ms	$>$500 ms

4.7 Qualitative Results

Figure 1 shows MNIST restoration samples, Figure 2 the training curve, and Figures 3–4 the main and ablation comparisons. Figure 5 shows the Set12 sliding-window outputs, which visibly darken toward MNIST statistics.

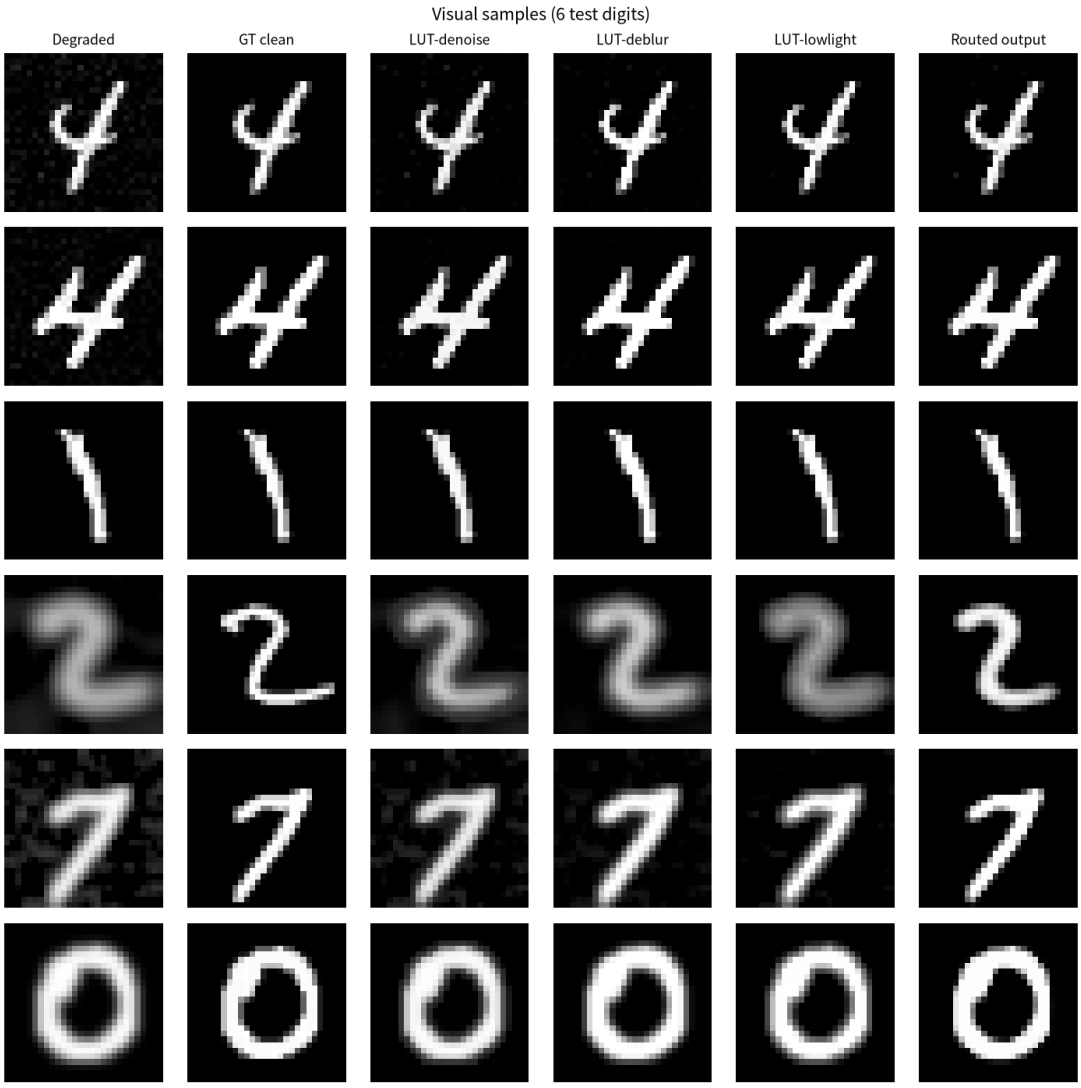


Figure 1: MNIST composite-degradation restoration: degraded input, ground truth, and EdgeLUT-Agent output.

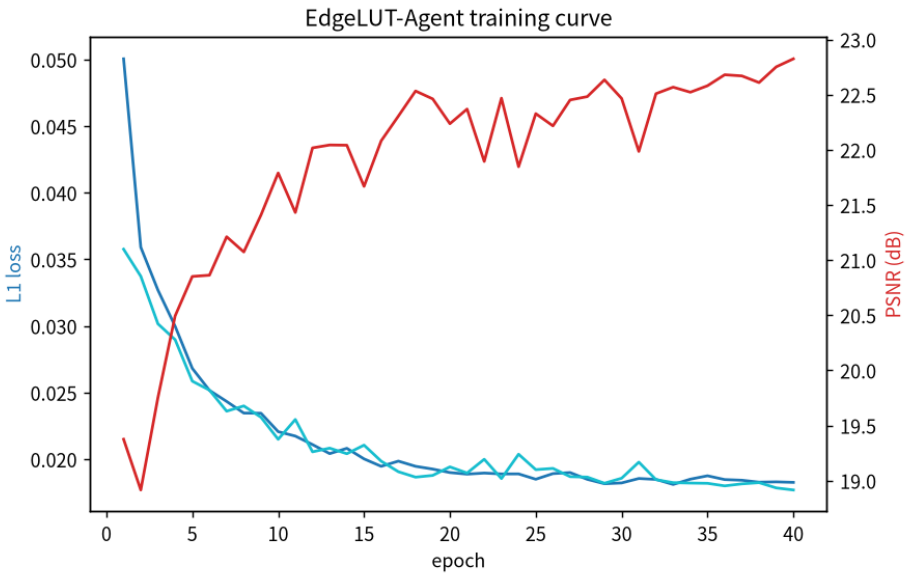


Figure 2: Training curve: PSNR on the validation split over 40 epochs.

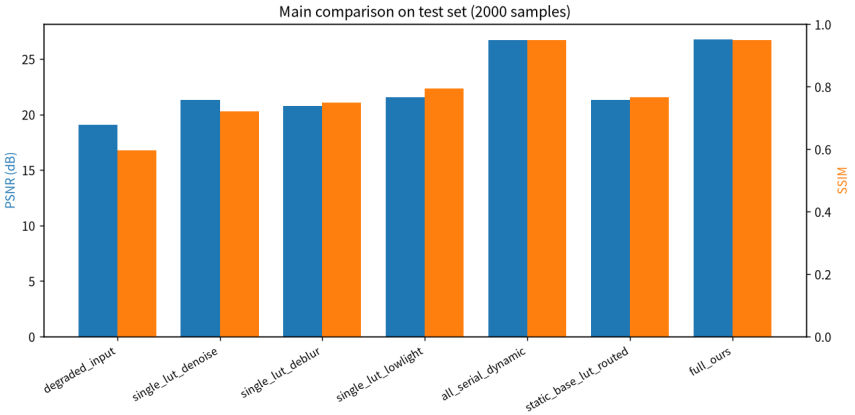


Figure 3: Per-method PSNR comparison on the MNIST test set.

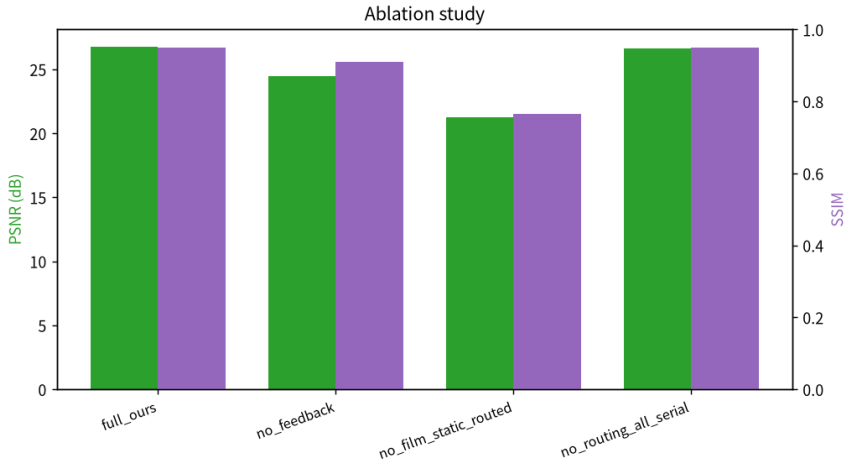


Figure 4: Ablation: contribution of FiLM conditioning, feedback re-route, and routing on/off.



Figure 5: Set12 sliding-window output. The model trained on 28×28 MNIST darkens 256×256 natural photos toward MNIST statistics, illustrating the domain gap.

5 Limitations and Future Work

- **Domain gap.** The model is trained and tested on 28×28 MNIST and fails on Set12 natural photos. Re-training on DIV2K [7] or a comparable high-resolution dataset is the obvious next step.
- **Qwen output collapse.** The 0.5B model cannot reliably perform numeric-reasoning routing. Exploring quantization-aware distillation, or a learned tiny router trained on the rule router's supervision, is more

promising than further prompting.

- **Feedback is heuristic.** The feedback thresholds (0.002, 0.3, 0.05) are hand-tuned. A learned router that predicts the need for re-routing directly would remove this manual knob.
- **No real-phone timing.** All CPU latency is measured on a 4-core laptop-class CPU; on-device measurements on ARM mobile cores are future work.

6 Conclusion

EdgeLUT-Agent demonstrates that a full agentic loop (perceive $\rightarrow$ reason $\rightarrow$ act $\rightarrow$ feedback) can be compressed into a 48,478-parameter, 194 KB, 0.99 ms CPU pipeline. The rule-based router is fast enough for real-time mobile deployment, while the Qwen2.5-0.5B comparison reveals a fundamental tension: on-device small LLMs are simultaneously too slow for per-image routing (64,000$\times$ slower than a threshold) and too small for reliable numeric reasoning (output collapse onto the few-shot template). Building the "agent" of an on-device restoration system is therefore not a matter of shrinking an existing LLM; it is a separate design problem, and the Dynamic 3D LUT family is a promising primitive on which to solve it.

References

[1] Zeng, H., Cai, J., Li, L., and Cao, Z. (2020). Learning 3D LUTs for Image Enhancement. *European Conference on Computer Vision (ECCV)*.

[2] Zhang, K., Zuo, W., Chen, Y., Meng, D., and Zhang, L. (2017). Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising. *IEEE Transactions on Image Processing*, 26(7):3142–3155.

[3] Tai, Y., Yang, J., Liu, X., and Shen, C. (2017). MemNet: A Persistent Memory Network for Image Restoration. *IEEE International Conference on Computer Vision (ICCV)*.

[4] Liu, H., Li, C., Wu, Q., and Lee, Y. J. (2023). Visual Instruction Tuning. *NeurIPS*.

[5] Qwen Team. (2024–2025). Qwen2.5 Technical Report. *arXiv:2412.15115*.

[6] Yang, S., et al. (2024). VisAgent: Large Language Models as Visual Planners for Vision-Language Tasks. *arXiv preprint*.

[7] Agustsson, E. and Timofte, R. (2017). NTIRE 2017 Challenge on Single Image Super-Resolution: Dataset and Study. *CVPR Workshops*.

[8] Abadi, M., et al. (2015–2019). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. *Google Brain*.

[9] Ma, X. and others. (2018). NCNN: A High-Performance Neural Network Inference Framework for Mobile Devices. *Tencent AI Lab*.

[10] Bai, J., et al. (2019). ONNX: Open Neural Network Exchange. *Microsoft / Facebook*.

[11] Dong, C., Loy, C. C., He, K., and Tang, X. (2014). Learning a Deep Convolutional Network for Image Super-Resolution. *ECCV*.

[12] Kim, J., Lee, J. K., and Lee, K. M. (2016). Accurate Image Super-Resolution Using Very Deep Convolutional Networks. *CVPR*.

[13] Zhang, Y., Tian, Y., Kong, Y., Zhong, B., and Fu, Y. (2018). Residual Dense Network for Image Super-Resolution. *CVPR*.

[14] Lim, B., Son, S., Kim, H., Nah, S., and Lee, K. M. (2017). Enhanced Deep Residual Networks for Single Image Super-Resolution. *CVPR Workshops*.

[15] Nah, S., Kim, T. H., and Lee, K. M. (2017). Deep Multi-Scale Convolutional Neural Networks for Dynamic Scene Deblurring. *CVPR*.

[16] Wei, C., Wang, W., Yang, W., and Liu, J. (2018). Deep Retinex Decomposition for Low-Light Enhancement. *BMVC*.

[17] Iglovikov, V., et al. (2020). Post-Training Quantization for On-Device Neural Networks. *arXiv preprint*.

[18] Howard, A. G., et al. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv:1704.04861*.